

DBMS COURSE PROJECT

Student Attendance & Leave Approval System

A MySQL relational database for daily attendance tracking, attendance percentage reporting and faculty-approved student leave

5

Tables

5

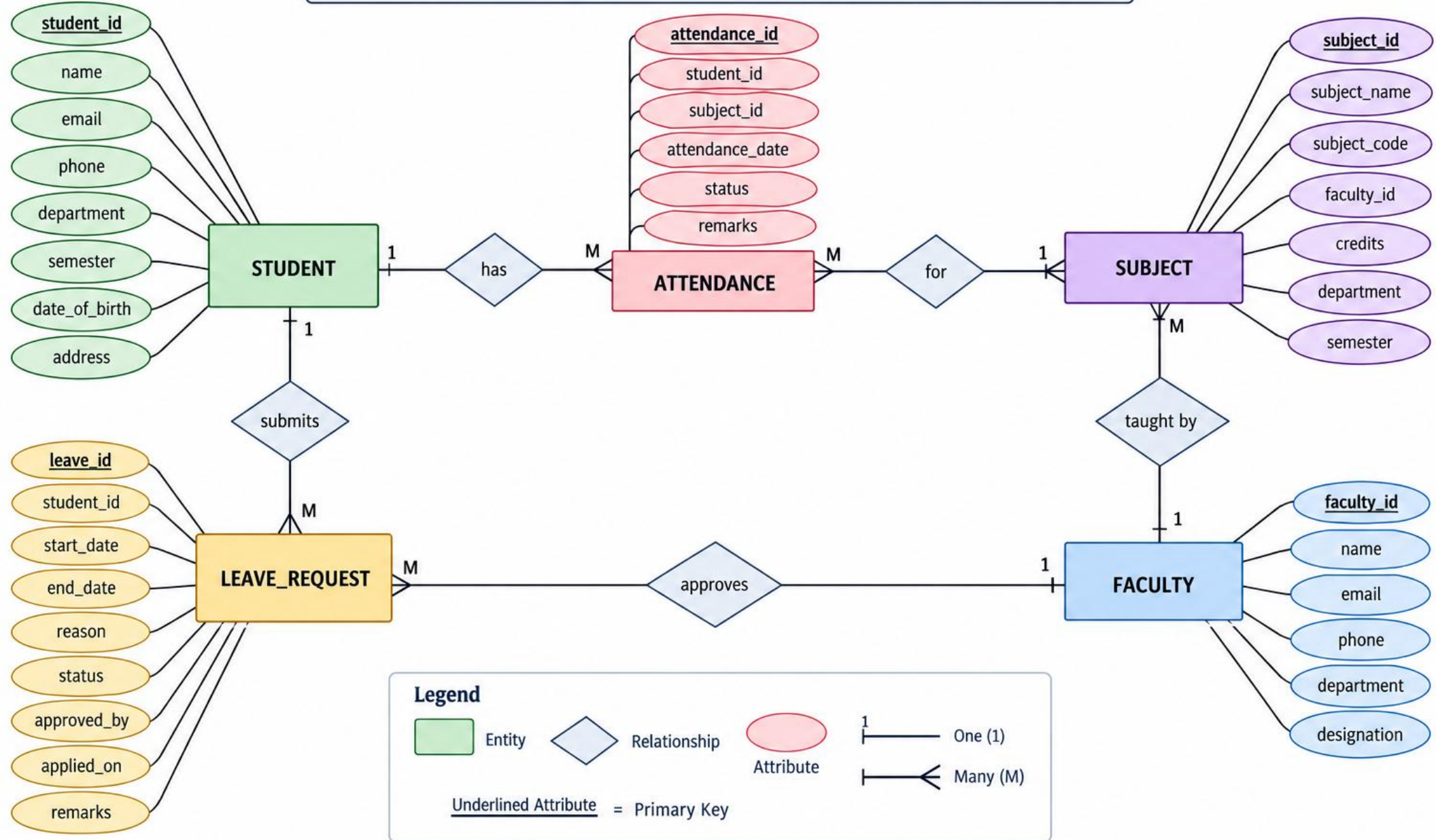
Relationships

MySQL

Platform

Student Attendance and Leave Approval System

ER Diagram (Crow's Foot Notation)



Every class.
Every absence.
Accounted for.

Sign in

Student

Faculty

Admin

Student ID

101

Password

Sign in

Demo credentials (for testing only)

Student: ID 101–120 · student123

Faculty: ID 201–215 · faculty123

Admin: admin · admin123

Project Overview

The system replaces manual attendance registers and paper leave slips with one centralised database. It delivers two major functions.

1 Attendance Management

- Store daily student attendance
- Record attendance subject-wise
- Identify present and absent students
- Calculate attendance percentages
- Find students below the required percentage

2 Leave Management

- Store student leave requests
- Track pending, approved and rejected requests
- Record which faculty processed the request
- View complete leave history per student

Objectives

1

Maintain student records

2

Maintain faculty records

3

Store subjects and assigned faculty

4

Record daily attendance

5

Calculate attendance percentages

6

Manage student leave requests

7

Allow faculty to approve or reject leave

8

Reduce manual record keeping

9

Provide simple SQL queries for reporting

Technology Used

1

MySQL

Relational database engine that stores every table and enforces the keys.

2

SQL

Language used for all table creation, inserts, joins and reports.

3

MySQL Workbench

Visual tool used to run the script, inspect tables and view results.

Database name

student_attendance

No external framework is required for the database version of this project.

The Five Database Tables

student

Personal and academic details of every enrolled student.

PK student_id

faculty

Details of every faculty member who teaches or approves leave.

PK faculty_id

subject

Subjects offered, each linked to the faculty who teaches it.

PK subject_id
FK faculty_id

attendance

One row per student, per subject, per class date.

PK attendance_id
FK student_id, subject_id

leave_request

Leave applications with their approval status.

PK leave_id
FK student_id, approved_by

student & faculty

student	
student_id	Unique student ID
name	Student name
email	Student email
phone	Student phone number
department	Student department
semester	Current semester
PRIMARY KEY student_id	

faculty	
faculty_id	Unique faculty ID
name	Faculty name
email	Faculty email
department	Faculty department
PRIMARY KEY faculty_id	

Both tables are pure master data — they hold no foreign keys, so they are created and populated first.

subject & attendance

subject	
subject_id	Unique subject ID
subject_name	Name of subject
faculty_id	Faculty teaching the subject
PRIMARY KEY subject_id	
FOREIGN KEY faculty_id -> faculty.faculty_id	

attendance	
attendance_id	Unique attendance record
student_id	Student being recorded
subject_id	Subject of the class
attendance_date	Date of class
status	Present or Absent
PRIMARY KEY attendance_id	
FOREIGN KEYS student_id, subject_id	

Business rule: a UNIQUE (student_id, subject_id, attendance_date) constraint blocks duplicate attendance for the same student in the same subject on the same day.

leave_request

leave_request	
leave_id	Unique leave request ID
student_id	Student requesting leave
start_date	Leave starting date
end_date	Leave ending date
reason	Reason for leave
status	Pending, Approved or Rejected
approved_by	Faculty who processed it
PRIMARY KEY leave_id	
FOREIGN KEYS student_id, approved_by	

Two foreign keys, two parents

```
student_id -> student.student_id
approved_by -> faculty.faculty_id
```

This single table records both who asked for leave and who decided on it.

Why approved_by allows NULL

A request that is still Pending has not been seen by any faculty member yet, so there is no approver to record. The column stays NULL until a decision is made.

Relationships

Student**1 : M****Attendance**

One student can have many attendance records.

Subject**1 : M****Attendance**

One subject can have many attendance records.

Faculty**1 : M****Subject**

One faculty member can teach multiple subjects.

Student**1 : M****Leave_Request**

One student can submit multiple leave requests.

Faculty**1 : M****Leave_Request**

A faculty member can process many leave requests.

The approved_by column identifies which faculty member processed each request.

Status Values

Leave request status

Pending

Submitted but not yet processed by any faculty member.

Approved

The faculty member has accepted the leave request.

Rejected

The faculty member has rejected the leave request.

Attendance status

Present

The student attended that subject's class on that date.

Absent

The student did not attend that class.

Both are stored as ENUM columns, so MySQL itself rejects any value outside this fixed list.

Installation & Setup

1

Install MySQL

Install MySQL Server together with MySQL Workbench.

2

Open Workbench

Connect to your local server and open a new SQL tab.

3

Open the script

File > Open SQL Script and choose student_attendance_leave.sql.

4

Execute all

Press the lightning-bolt button, or Ctrl + Shift + Enter.

The script then runs automatically, in order:

● Creates the database

● Selects the database

● Creates all five tables

● Adds primary keys

● Adds foreign keys

● Inserts faculty, students & subjects

● Inserts 160 attendance records

● Inserts 10 leave requests

Basic Queries

Simple SELECT statements to confirm every table loaded correctly.

```
-- View all master data
SELECT * FROM student;

SELECT * FROM faculty;

SELECT * FROM subject;

-- View the transaction tables
SELECT * FROM attendance;

SELECT * FROM leave_request;
```

What you should see

8 students · 4 faculty · 5 subjects · 160 attendance rows · 10 leave requests

Why 160 attendance rows?

8 students × 5 subjects × 4 class dates = 160.

Enough data for the percentage and shortage reports to return meaningful results.

Attendance Reports

```
-- Attendance percentage for every student
SELECT s.name,
       COUNT(*) AS total_classes,
       SUM(a.status = 'Present') AS attended,
       ROUND(SUM(a.status = 'Present')*100.0
             / COUNT(*), 2) AS percentage
FROM student s
JOIN attendance a ON s.student_id = a.student_id
GROUP BY s.student_id, s.name;
```

```
-- Shortage list: below the 75% requirement
SELECT s.name,
       ROUND(SUM(a.status='Present')*100.0
             / COUNT(*), 2) AS percentage
FROM student s
JOIN attendance a ON s.student_id = a.student_id
GROUP BY s.student_id, s.name
HAVING percentage < 75;
```

Sample output

Aarav Sharma

95%

Karthik Reddy

90%

Diya Patel

85%

Meera Joshi

80%

Sneha Nair

75%

Rohan Verma

70%

Ananya Iyer

60%

Vikram Singh

55%

Three students fall below 75% and appear on the shortage list.

Leave Management Queries

```
-- Requests still waiting for a decision
SELECT l.leave_id, s.name,
       l.start_date, l.end_date, l.reason
FROM leave_request l
JOIN student s ON l.student_id = s.student_id
WHERE l.status = 'Pending';
```

```
-- Approve a request
UPDATE leave_request
SET    status = 'Approved',
       approved_by = 101
WHERE  leave_id = 2;
```

```
-- Reject a request
UPDATE leave_request
SET    status = 'Rejected',
       approved_by = 102
WHERE  leave_id = 6;
```

```
-- Full leave history with approver
SELECT s.name, l.start_date, l.end_date,
       l.reason, l.status, f.name
FROM leave_request l
JOIN student s ON l.student_id=s.student_id
LEFT JOIN faculty f
      ON l.approved_by = f.faculty_id;
```

Example Workflow

Recording attendance

- 1 Faculty opens the class session
- 2 Faculty marks each student
- 3 Attendance record is created
- 4 Row stored in the attendance table
- 5 Percentage report is generated

Approving leave

- 1 Student submits a leave request
- 2 Request status is set to Pending
- 3 Faculty reviews the request
- 4 Faculty approves or rejects it
- 5 Status and approved_by are updated

Sample Use Case

Aarav Sharma applies for two days of medical leave.

Pending

Student	Aarav Sharma
Start date	2026-09-10
End date	2026-09-11
Reason	Medical reason
Approved by	NULL



Approved

Student	Aarav Sharma
Start date	2026-09-10
End date	2026-09-11
Reason	Medical reason
Approved by	Dr. Amit Sharma

```
UPDATE leave_request SET status = 'Approved', approved_by = 101 WHERE leave_id = 1;
```

Advantages

1

Simple design

A clean five-table relational model that is easy to explain and defend.

2

Less manual work

Replaces paper registers and handwritten leave slips.

3

Centralised leave

Every request and decision lives in one table.

4

Instant reporting

Attendance percentages are calculated by a single query.

5

Data consistency

Primary and foreign keys prevent orphaned records.

6

Extensible

Can grow into a full web or mobile application.

Future Enhancements

1 Security & access

- Student, faculty and admin login
- Role-based access control
- Password authentication
- Audit logs

2 Features

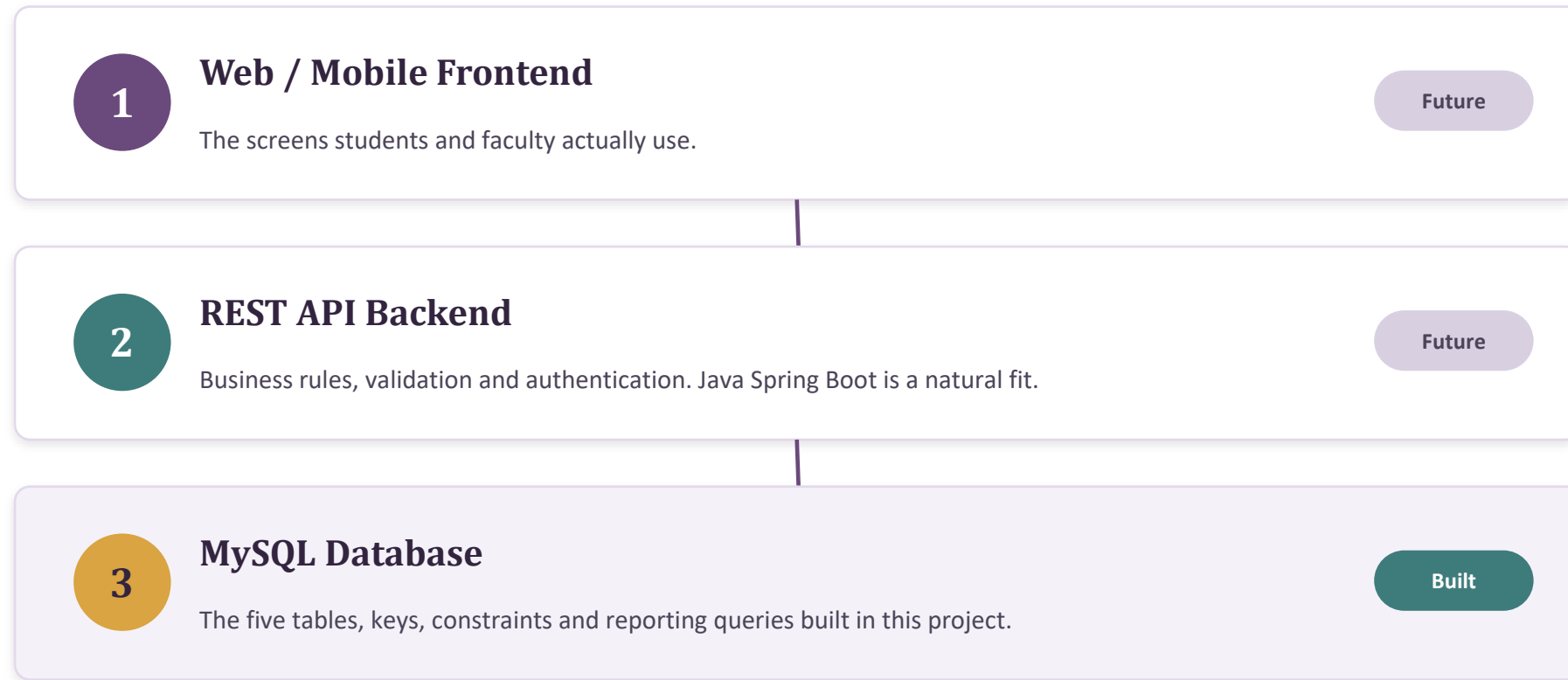
- Email notifications
- Leave document uploads
- Attendance percentage alerts
- Multiple faculty approvals

3 Reporting & reach

- Monthly attendance reports
- Dashboard and charts
- Department-wise reports
- REST API, web & mobile apps

Suggested Application Architecture

The current project delivers the database layer. A full-stack build would add two layers above it, without changing the core schema.



Conclusion

The system provides a simple relational database solution for managing student attendance and leave applications, and can serve as the foundation for a larger college management application.

DBMS concepts demonstrated

- Database creation

- Table creation

- Primary keys

- Foreign keys

- One-to-many relationships

- INSERT operations

- SELECT queries

- JOIN operations

- UPDATE operations

- GROUP BY

- HAVING

- Aggregate functions

Thank you

M. Pranay Kumar Reddy · 25WU0102138